2026 학년도 졸업프로젝트

중 간 보 고 서

스케줄링 알고리즘을 활용한
다중 사용자 LLM API 요청 관리 시스템

학 과	홍익대학교 컴퓨터공학과
학 번	C235180
성 명	서민지
지도교수	이장호 교수님

2026 년 4 월 12 일

홍익대학교 컴퓨터공학과

1. 서론

1.1 연구 배경

최근 ChatGPT, Claude, Gemini 등 대규모 언어 모델(Large Language Model, LLM) API 서비스가 다양한 분야에서 활용되고 있다. 이에 따라 기업이나 연구실에서도 하나의 조직 계정으로 여러 구성원이 LLM API 를 사용하거나, 자체 LLM 서버를 구축하여 여러 사용자에게 서비스하는 사례가 늘어나고 있다.

이러한 다중 사용자 환경에서는 요청을 어떻게 관리할지가 문제가 된다. OpenAI 는 조직(Organization) 단위로 분당 요청 수와 토큰 수에 한도를 두고 있고 [1], Anthropic 도 사용 등급별로 호출 한도를 적용하고 있다 [2]. 이러한 한도는 조직이나 계정 단위로 적용되므로, 같은 조직에 속한 구성원들이 한정된 호출 한도를 어떻게 나누어 쓸지는 별도로 관리해야 하는 문제이다. 자체 LLM 서버를 운영하는 경우에도, 동시에 처리할 수 있는 요청 수가 제한되어 같은 문제가 발생한다.

별도의 관리 기능이 없다 보니, 요청은 도착한 순서대로 처리되거나 호출 빈도 제한 수준에서 관리되는 것이 일반적이다. 이 경우 긴급한 요청도 순서를 기다려야 하고, 사용자별로 자원을 다르게 배분하기 어려우며, 자원이 공정하게 나누어지고 있는지 확인하기도 쉽지 않다.

1.2 연구 목적

운영체제 수업에서 프로세스 스케줄링 알고리즘을 배우면서, "이 알고리즘들을 LLM API 요청 관리에 적용하면 어떨까?"라는 궁금증에서 이 프로젝트를 시작하게 되었다. 스케줄링 이론을 LLM API 환경에 적용해보고, 알고리즘별 성능과 공정성을 비교해보는 것이 본 프로젝트의 목적이다.

구체적으로 다음 세 가지를 수행하고자 한다.

스케줄링 알고리즘 적용: FCFS, Priority Scheduling, MLFQ, WFQ 의 네 가지 스케줄링 알고리즘을 LLM API 환경에 맞게 구현한다.

알고리즘별 성능 비교: 평균 대기시간, 처리량 등의 지표로 각 알고리즘의 효율성을 비교한다.

공정성 분석: Jain's Fairness Index(JFI)를 활용하여, 각 알고리즘이 사용자 간 자원을 얼마나 공정하게 배분하는지 분석한다.

1.3 진행 상황

현재까지 시스템의 핵심 기능을 구현하고, 기본적인 실험을 수행하였다. 네 가지 스케줄링 알고리즘 모두 동작하는 상태이며, 각 알고리즘의 단위 테스트를 통해 정상 동작을 확인하였다. 이후 추가 실험과 분석을 진행하고 있다. 이 보고서에서는 시스템 설계, 구현 현황, 중간 실험 결과를 정리한다.

2. 관련 연구

이 장에서는 본 프로젝트에서 활용한 스케줄링 알고리즘의 이론적 배경, 기존 LLM 서빙 도구의 현황, 그리고 공정성 측정 방법을 살펴본다.

2.1 스케줄링 알고리즘

프로세스 스케줄링은 운영체제에서 CPU 자원을 여러 프로세스에 나누어 주기 위한 방법이다. 어떤 순서로 처리할지에 따라 다양한 알고리즘이 연구되어 왔다 [3].

FCFS(First-Come, First-Served)는 요청이 도착한 순서대로 처리하는 가장 단순한 방식이다. **구현이** 간단하지만, 처리 시간이 긴 요청이 앞에 있으면 뒤의 짧은 요청들까지 오래 기다리게 **되는** 호위 효과(Convoy Effect)가 발생할 수 있다 [3].

Priority Scheduling 은 각 프로세스에 긴급도(우선순위)를 부여하여 더 급한 것을 먼저 처리하는 방식이다. 다만 우선순위가 낮은 프로세스가 계속 밀려서 영원히 처리되지 못하는 기아 현상(Starvation)이 발생할 수 있다. 이를 막기 위해 오래 기다린 프로세스의 우선순위를 점차 올려주는 에이징(Aging) 기법이 사용된다 [3].

MLFQ(Multi-Level Feedback Queue)는 여러 개의 대기열을 두고, 작업의 특성에 따라 우선순위를 자동으로 조정하는 알고리즘이다. 새 작업은 가장 높은 우선순위 큐에 들어가고, 시간 할당량 안에 끝나지 않으면 한 단계 낮은 큐로 내려간다. 짧은 작업은 빠르게, 긴 작업은 천천히 처리되는 효과가 있다 [3].

WFQ(Weighted Fair Queuing)는 네트워크 분야에서 사용되는 공정한 자원 배분 알고리즘이다. 각 사용자에 가중치를 부여하고, 가중치가 클수록 더 많은 자원을 받도록 설계되어 있다. 구체적인 동작 방식은 4.3 절에서 설명한다.

2.2 LLM 서빙 시스템

LLM 서빙이란 학습이 완료된 LLM 을 사용자에게 제공하는 것을 뜻한다. 대표적인 오픈소스 도구로 vLLM [4]과 Hugging Face TGI [5]가 있다. vLLM 은 PagedAttention 기반의 높은 처리 성능을, TGI 는 배치 처리(Continuous Batching) 기능을 제공하지만, 두 도구 모두 다중 사용자 환경에서의 요청 우선순위 관리나 공정성 보장 기능은 공식 문서에서 확인하지 못하였다. 본 프로젝트는 이 부분에 스케줄링 이론을 적용해보려고 한다.

2.3 공정성 측정

여러 사용자가 자원을 공유하는 시스템에서, 자원이 얼마나 고르게 나뉘었는지를 측정하는 지표로 Jain's Fairness Index(JFI)가 있다 [6]. JFI 는 자원이 얼마나 공정하게 배분되었는지를 0 에서 1 사이 점수 하나로 나타내는 지표이다. 1 에 가까울수록 공평하게 나뉜 것이고, 특정 사용자만 자원을 많이 받으면 0 에 가까워진다.

$$JFI = (x_1 + x_2 + \dots + x_n)^2 / (n \times (x_1^2 + x_2^2 + \dots + x_n^2))$$

여기서 x_i는 사용자 i 가 할당받은 자원량, n 은 사용자 수이다. 모든 사용자가 동일하게 자원을 받으면 JFI = 1 이고, 한 명이 독점하면 JFI = 1/n 에 가까워진다. 네트워크 대역폭 배분, CPU 스케줄링 등

다양한 자원 공유 문제에서 사용되며, 본 프로젝트에서도 이 지표를 활용하여 스케줄링 알고리즘의 공정성을 비교하였다.

3. 시스템 설계

이 장에서는 제안서에서 설명한 시스템의 구체적인 설계를 정리한다. 전체 아키텍처, 모듈 간 관계, 그리고 데이터 처리 흐름을 살펴본다.

3.1 설계 개요

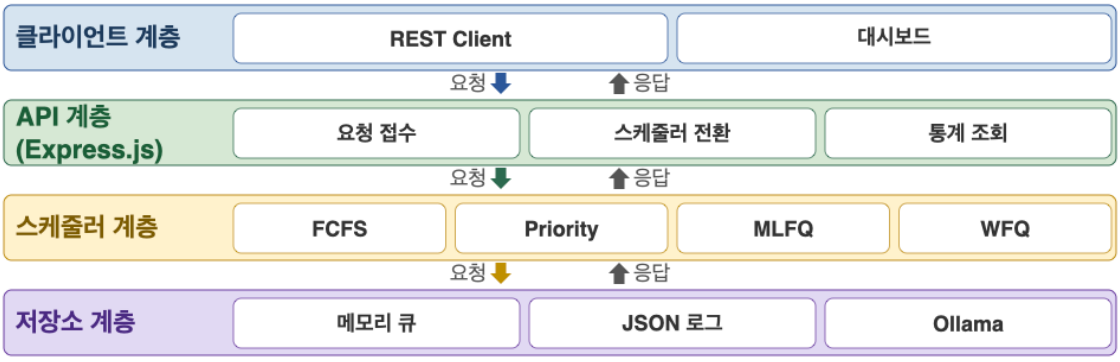
본 시스템의 기본 아이디어는, OS 에서 프로세스가 CPU 를 할당받기 위해 대기하는 것처럼, LLM API 요청도 대기열에서 순서를 기다리는 구조를 만드는 것이다. 표 1 은 OS 개념과 LLM 환경 사이의 대응 관계를 정리한 것이다.

표 1. OS 개념과 LLM 환경 대응 관계

OS 개념	LLM 환경 대응	설명
프로세스	LLM API 요청	스케줄링의 기본 단위
CPU 시간	LLM 추론 시간	할당되는 자원
준비 큐	요청 대기열	처리를 기다리는 작업의 저장소
우선순위	요청 우선순위	개별 요청의 긴급도 (Priority Scheduling 에서 사용)
(해당 없음)	사용자 등급	사용자별 서비스 수준 (WFQ 에서 사용)
시간 할당량	큐별 처리 시간 한도	MLFQ 에서 큐 이동 기준

3.2 시스템 아키텍처

시스템은 4 개의 계층으로 구성하였다 (그림 1).



[그림 1] 시스템 아키텍처

클라이언트 계층: 사용자가 REST API(HTTP 요청으로 데이터를 주고받는 웹 표준 방식)를 통해 LLM 요청을 보내고, 처리 상태를 조회하는 부분이다.

API 계층: Express.js [8] 기반의 HTTP 서버로, 요청 접수, 스케줄러 전환, 통계 조회 등의 기능을 제공한다. 사용자의 요청을 받아서 요청 우선순위와 사용자 등급 정보를 포함한 요청 객체를 생성하고, 이를 스케줄러에 전달하는 역할을 한다.

스케줄러 계층: 시스템의 핵심 부분으로, 네 가지 스케줄링 알고리즘이 구현되어 있다. 모든 알고리즘은 같은 인터페이스(enqueue, dequeue)를 따르기 때문에, 서버 시작 시 환경변수 하나로 알고리즘을 선택할 수 있다. 사용자 등급에 따른 가중치 결정도 스케줄러 계층에서 처리한다.

저장소 계층: 메모리 큐(외부 데이터베이스 없이 프로그램 메모리의 배열로 관리하는 대기열)로 요청을 관리하고, JSON 파일로 처리 이력을 기록한다. LLM 호출은 Ollama [7]를 통해 로컬에서 수행한다.

3.3 설계 방침

스케줄링 알고리즘을 LLM 환경에 그대로 적용할 수는 없으므로, 다음과 같은 설계 방침을 정하였다.

첫째, LLM 추론은 한번 시작되면 중간에 멈추기 어렵기 때문에, 실제 서버에서는 비선점형 방식을 사용한다. 하나의 요청이 LLM 처리를 마치면, 스케줄러가 대기열에서 다음에 처리할 요청을 골라주고, 선택된 요청은 완료될 때까지 중단 없이 처리된다.

다만 MLFQ 의 경우, 비선점형으로 운영하면 요청이 중단 없이 끝까지 처리되어 큐 간 이동이 발생하지 않으므로, 짧은 요청을 우선 처리하는 핵심 효과가 나타나지 않았다 (5.2 절 참조). 이것이 MLFQ 알고리즘 자체의 한계인지, LLM 환경의 제약 때문인지 확인하기 위해 선점형 모드를 시뮬레이션 환경에서 추구 구현하였다 (5.3 절).

둘째, 시스템에서 사용하는 등급 체계는 두 가지이며, 각각 다른 스케줄러에서 사용된다 (표 2).

표 2. 등급 체계 비교

구분	요청 우선순위	사용자 등급
적용 대상	개별 요청	사용자(테넌트)
등급	URGENT, HIGH, NORMAL, LOW	Enterprise, Premium, Standard, Free
성격	요청마다 다를 수 있음	사용자에게 고정
사용처	Priority Scheduling	WFQ

요청 우선순위는 "이 요청이 얼마나 급한가"를 나타내고, 사용자 등급은 "이 사용자가 얼마만큼의 자원을 받을 자격이 있는가"를 나타낸다. 두 개념은 독립적이므로, Enterprise 등급 사용자가 LOW 우선순위 요청을 보내는 것도 가능하다. FCFS 와 MLFQ 는 이러한 등급과 무관하게 동작한다.

셋째, 각 알고리즘의 핵심 원리는 유지하면서 스케줄링 기준은 LLM 환경에 맞게 조정하였다.

Priority Scheduling: 요청 우선순위(URGENT, HIGH, NORMAL, LOW)에 따라 긴급한 요청을 먼저 처리한다. 오래 기다린 요청의 우선순위를 높여주는 에이징(Aging) 기법도 구현하였다(5 초 주기).

MLFQ: 4 단계 큐(Q0~Q3)를 두고, 큐별로 다른 시간 할당량을 적용한다. OSTEP [3]의 표준 MLFQ 규칙을 따르며, 기아 방지를 위해 5 초마다 모든 요청을 최상위 큐로 올려준다.

WFQ: 사용자 등급(Enterprise~Free)에 따라 가중치(100, 50, 10, 1)를 부여하고, 가중치를 반영한 가상 완료 시각을 계산하여 서비스를 차등 제공한다.

3.4 데이터 처리 흐름

사용자가 요청을 보내면 다음 순서로 처리된다 (그림 2).

[그림 2] 데이터 흐름도

1. 사용자가 REST API(POST /api/requests)로 요청을 보낸다.
2. API 계층에서 입력을 확인하고, 요청 객체를 생성한다.
3. 생성된 요청을 스케줄러의 대기열에 등록한다(enqueue).
4. 처리 요청이 들어오면, 스케줄러는 선택된 알고리즘에 따라 대기열에서 다음에 처리할 요청을 꺼낸다(dequeue).
5. 선택된 요청을 Ollama 를 통해 LLM 에 전달하고, 응답을 기다린다.
6. LLM 응답이 돌아오면 결과를 클라이언트에 반환하고, 처리 이력을 JSON 파일에 기록한다.

대기열에 요청이 남아 있으면 4 번부터 반복한다.

4. 구현 현황

4.1 기술 스택

표 3. 기술 스택

항목	기술/도구	설명
런타임	Node.js 22 LTS [9]	JavaScript 실행 환경
프레임워크	Express.js 4.18 [8]	REST API 서버
언어	JavaScript (ES2024)	서버 및 스케줄러 구현 언어
LLM	Ollama (로컬) [7]	로컬에서 REST API 로 LLM 을 호출할 수 있는 도구
저장소	메모리 큐 + JSON 파일	외부 데이터베이스 없이 구현
의존성	1 개 (express)	외부 라이브러리는 Express.js 만 사용

외부 라이브러리를 최소한으로 사용하고, 스케줄링 알고리즘을 직접 구현하여 학습 효과를 높이는 것을 목표로 하였다.

4.2 코드 구조

소스 코드는 api, schedulers, queue, storage, llm 의 5 개 모듈로 구성되어 있다. 핵심은 schedulers 모듈로, BaseScheduler 를 상속받는 4 개 스케줄러(FCFS, Priority, MLFQ, WFQ)가 각각 `enqueue()`와 `dequeue()` 메서드를 구현한다. 같은 인터페이스를 사용하므로, 서버 코드를 수정하지 않고 환경변수만으로 알고리즘을 전환할 수 있다.

4.3 핵심 알고리즘 구현

4 개 알고리즘 중 FCFS 와 Priority 는 구현이 단순하므로, 여기서는 상대적으로 복잡한 MLFQ 와 WFQ 의 핵심 동작만 설명한다.

MLFQ 선점 처리

MLFQ 의 핵심은 짧은 요청을 빠르게 처리하고, 긴 요청은 낮은 우선순위로 내리는 것이다. 500ms 간격으로 요청 상태를 확인하여, 시간 할당량을 초과한 요청은 하위 큐로 이동시키고 상위 큐의 다음 요청을 먼저 처리한다. 선점 시 사용 시간은 0 으로 리셋되어, 새 큐의 할당량을 다시 받는다. MLFQ 는 OSTEP [3]의 5 가지 규칙(표 4)에 따르며, 큐별 시간 할당량은 표 5 과 같다.

표 4. MLFQ 규칙

규칙	설명
Rule 1	높은 큐의 요청이 낮은 큐보다 먼저 처리된다
Rule 2	같은 큐 안에서는 도착 순서대로 처리한다
Rule 3	새 요청은 항상 최상위 큐(Q0)에 들어간다
Rule 4	시간 할당량을 다 쓰면 한 단계 아래 큐로 내려간다
Rule 5	일정 시간마다 모든 요청을 Q0 로 올려준다 (기아 방지)

표 5. MLFQ 큐별 시간 할당량

큐 레벨	시간 할당량	대상
Q0 (최상위)	1,000ms	새로 들어온 요청 (짧은 요청)
Q1	3,000ms	중간 길이 요청
Q2	8,000ms	긴 요청
Q3 (최하위)	무제한	아주 긴 요청

Rule 5 의 부스팅(Boosting)은 5 초마다 모든 요청을 최상위 큐(Q0)로 올려주는 것으로, 하위 큐에 오래 머물러 있는 요청이 영원히 처리되지 못하는 것을 방지한다.

WFQ 가상 완료 시각 계산

WFQ 에서는 각 요청의 가상 완료 시각(Virtual Finish Time, VFT)을 계산하여 이 값이 가장 작은 요청을 먼저 처리한다. VFT 는 다음과 같이 계산된다.

$$VFT_i = VFT_{(i-1)} + 1 / 가중치$$

여기서 처리 비용은 요청 1 건당 1 로 고정하였다. LLM 응답의 출력 토큰 수는 생성이 완료되기 전까지 알 수 없고, 입력 토큰만으로는 실제 자원 소모를 정확히 반영하기 어렵다. 또한 공정성 측정(JFI)도 처리 건수 기준이므로, 스케줄링과 측정의 기준을 요청 건수로 일치시켰다.

가중치가 큰 사용자는 VFT 가 느리게 증가하므로, 더 자주 선택되어 더 많은 자원을 받게 된다. 예를 들어, Enterprise(가중치 100)와 Free(가중치 1)가 요청을 동시에 보내면, Enterprise 의 VFT 증가분은 1/100 = 0.01 이고 Free 는 1/1 = 1 이 되어, Enterprise 의 요청이 먼저 처리된다. 이를 통해 가중치에 비례한 서비스가 이루어진다.

4.4 구현 결과 요약

현재까지 핵심 기능을 구현하고 테스트를 완료하였다.

스케줄러: 4 개 알고리즘 모두 동작 확인 (FCFS, Priority, MLFQ, WFQ)

테스트: 각 스케줄러별 단위 테스트를 작성하여 정상 동작을 확인하였다

알고리즘 선택: 서버 시작 시 환경변수(SCHEDULER_TYPE)로 스케줄링 알고리즘을 선택할 수 있다

5. 중간 실험 결과

5.1 실험 환경

모든 실험은 실제 LLM 을 호출하지 않는 시간 기반 시뮬레이션 방식으로 수행하였다. 각 요청에 처리 시간을 미리 지정하고, 스케줄러가 선택한 순서대로 한 건씩 순차 처리하는 방식이다. 이렇게 하면 네트워크 지연이나 LLM 상태에 따른 변동 없이, 알고리즘 자체의 스케줄링 특성만 비교할 수 있다.

표 6. 실험 설정 비교

항목	기본 실험	추가 실험 (다중 시드)
목적	4 개 알고리즘 동일 조건 비교	MLFQ 선점형 효과 검증
요청 수	100 건	500 건 × 5 회 반복
사용자 수	4 명 (등급별 1 명)	구분 없음
사용자 등급	Enterprise, Premium, Standard, Free	해당 없음
요청 할당	사용자에게 라운드 로빈	해당 없음

우선순위 분포	LOW 30%, NORMAL 40%, HIGH 20%, URGENT 10%	해당 없음
처리 시간	10~100ms (균일 랜덤)	Short 100~800ms, Medium 1.2~4 초, Long 5~10 초
도착 패턴	순차 도착 (평균 약 5ms 간격)	버스트 도착 (20 건 동시 × 25 회, 2 초 간격)
처리 방식	순차 처리 (한 번에 1 건)	순차 처리 (한 번에 1 건)
MLFQ 모드	비선점형	선점형 (타임 슬라이스 500ms)
재현성	고정 시드 1 개	5 개 랜덤 시드

기본 실험은 네 가지 알고리즘을 동일 조건에서 비교하기 위한 것이고, 추가 실험은 5.2 절에서 비선점형 MLFQ 가 FCFS 와 동일한 결과를 보인 뒤, 선점형 MLFQ 의 효과를 검증하기 위해 설계하였다.

5.2 기본 실험 결과 (100 건)

네 가지 알고리즘을 같은 100 건 요청에 대해 실행한 결과는 표 7 과 같다.

표 7. 알고리즘별 성능 비교 (100 건)

스케줄러	평균 대기시간	처리량(req/s)	핵심 발견
FCFS	2,572ms	18.0	베이스라인 (도착 순서 처리)
Priority	2,677ms	17.1	URGENT: 약 42ms, LOW: 약 4,600ms
MLFQ	2,572ms	18.0	비선점형에서는 FCFS 와 동일
WFQ	2,476ms	18.0	Enterprise: 약 430ms, Free: 약 4,540ms

Priority Scheduling: 우선순위에 따른 대기시간 차이가 뚜렷하게 나타났다. URGENT 요청은 평균 약 42ms 로 거의 즉시 처리된 반면, LOW 요청은 약 4,600ms 로 크게 늘어났다. 전체 평균이 FCFS 보다 약간 높은 것은, 높은 우선순위 요청을 먼저 처리한 만큼 낮은 우선순위의 대기가 길어지기 때문이다. 에이징 효과는 요청당 처리 시간이 짧아(10~100ms) 이번 실험에서는 관측되지 않았으며, 실서버 실험에서 검증할 계획이다 (6 절 참조).

WFQ: Enterprise 등급(가중치 100)의 평균 대기시간은 약 430ms, Free 등급(가중치 1)은 약 4,540ms 로 약 10 배 차이가 나타나, 가중치에 비례한 차등 서비스를 확인하였다. JFI 는 0.32 로 낮게 나왔는데, 이는 가중치가 다른 사용자에게 동일 건수를 배정한 실험 설계의 영향이다. 같은 등급 내에서는 대기시간 편차가 작아 등급 내 배분은 고르게 이루어졌다.

MLFQ: 비선점형에서는 FCFS 와 동일한 결과가 나왔다 (평균 대기시간 2,572ms, 처리량 18.0 req/s). 모든 요청이 중단 없이 완료되어 큐 간 이동이 발생하지 않기 때문이며, 이 결과가 3.3 절의 선점형 시뮬레이션을 추가 구현한 계기이다.

5.3 MLFQ 선점형 실험 결과

MLFQ 의 이론적 효과를 검증하기 위해, 선점형 시뮬레이션을 추가로 수행하였다 (표 6 의 추가 실험 조건). 요청 수 500 건, 처리 시간 최대 10 초, 버스트 도착 구조로 기본 실험보다 규모를 크게 늘렸으며, 5 개 시드로 반복하였다. 선점형에서는 요청의 중단과 재개가 반복되므로, 지표를 응답시간(도착~완료)으로 측정하였다.

표 8. MLFQ 선점형 vs FCFS 비교 (요청 유형별)

요청 유형	처리 시간 범위	FCFS	MLFQ (선점형)	변화
Short	100~800ms	약 635 초	약 170 초	73% 감소
Medium	1.2~4 초	약 645 초	약 729 초	13% 증가
Long	5~10 초	약 650 초	약 1,226 초	89% 증가

요청 유형은 처리 시간에 따라 분류하였다. Short 는 Q0 시간 할당량(1,000ms) 이내에 완료되는 요청, Medium 은 Q1(3,000ms)이 필요한 요청, Long 은 Q2(8,000ms) 이상이 필요한 요청이다. 분포는 Short 40%, Medium 40%, Long 20%이다.

선점형 MLFQ 에서 Short 유형 요청의 응답시간이 FCFS 대비 약 73% 줄어든 것을 확인하였다. 여러 번 반복해도 비슷한 결과가 나와, 한 번만의 결과가 아니라는 것을 확인하였다.

반면, Long 유형 요청의 응답시간은 MLFQ 에서 더 늘어났는데, 이는 Short 유형 요청을 먼저 처리하기 위해 Long 유형 요청이 하위 큐로 밀려나기 때문이다. 이처럼 MLFQ 는 Short 유형 요청의 응답시간을 크게 개선하지만, Long 유형 요청에는 불리한 면이 있다. 다만 Rule 5 의 부스팅이 주기적으로 작동하여, Long 유형 요청이 영원히 처리되지 않는 상황은 방지된다.

6. 향후 계획

현재까지 네 가지 스케줄링 알고리즘의 기본 구현과 중간 실험을 완료하였다. 남은 학기 동안 다음 작업을 추가로 수행할 계획이다.

표 9. 남은 일정

기간	주요 활동	산출물
4 월 중순~4 월 말	실서버 실험 (Ollama), 실험 규모 확대	추가 실험 코드 및 데이터
5 월 초~5 월 중순	공정성 분석, 결과 정리, 최종보고서 작성	최종보고서
5 월 말	발표 자료 준비, 시스템 데모 구성	발표 PPT, 데모

구체적으로는 다음을 계획하고 있다.

실서버 실험: 현재까지의 실험은 처리 시간을 미리 지정한 시뮬레이션으로 수행하였다. Ollama 를 활용하여 실제 LLM 을 호출하는 환경에서 FCFS, Priority, WFQ 세 알고리즘의 성능을 비교한다. 실서버에서는 추론 중단이 불가능하므로 비선점형으로만 운영되며, 비선점형 MLFQ 는 5.2 절에서

확인한 대로 FCFS 와 동일한 결과가 예상되므로 실서버 비교 대상에서 제외한다. 또한 실제 LLM 추론은 수 초 이상 소요되므로, 기본 실험에서 관측하지 못한 Priority Scheduling 의 에이징 효과도 함께 검증한다.

선점형 MLFQ 시뮬레이션 확대: 5.3 절의 선점형 MLFQ 실험을 1,000 건 이상의 요청으로 확대하고, 버스트 트래픽 등 다양한 시나리오를 추가한다. 선점형 환경에서 요청 유형별 응답시간 차이가 일관되게 나타나는지 확인한다.

공정성 분석: 현재 JFI 는 WFQ 에서만 측정하였으나, 네 가지 알고리즘 모두에 대해 사용자별 대기시간 기준으로 JFI 를 계산하여 비교한다. 또한 WFQ 의 이중 JFI(시스템 수준, 등급 내 수준)를 활용한 공정성 분석을 더 자세히 진행하고, 알고리즘 간 공정성과 성능의 장단점을 비교한다.

데모 시스템 구성: 최종 발표를 위한 실시간 데모 환경을 준비하여, 실제로 요청을 보내면서 알고리즘별 차이를 보여줄 수 있도록 한다.

참고문헌

[1] OpenAI, "Rate limits," OpenAI API Documentation. [온라인] Available: https://platform.openai.com/docs/guides/rate-limits (접속: 2026-03-14)

[2] Anthropic, "Rate limits," Anthropic API Documentation. [온라인] Available: https://docs.anthropic.com/en/api/rate-limits (접속: 2026-03-14)

[3] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, "Operating Systems: Three Easy Pieces," v1.10, Arpaci-Dusseau Books, 2023. [온라인] Available: https://pages.cs.wisc.edu/~remzi/OSTEP/

[4] vLLM Project, "vLLM: Easy, Fast, and Cheap LLM Serving." [온라인] Available: https://docs.vllm.ai/ (접속: 2026-03-14)

[5] Hugging Face, "Text Generation Inference (TGI)." [온라인] Available: https://huggingface.co/docs/text-generation-inference/ (접속: 2026-03-14)

[6] Wikipedia, "Fairness measure — Jain's fairness index." [온라인] Available: https://en.wikipedia.org/wiki/Fairness_measure (접속: 2026-04-09)

[7] Ollama, "Ollama Documentation." [온라인] Available: https://ollama.com/ (접속: 2026-03-14)

[8] Express.js, "Express - Node.js Web Application Framework." [온라인] Available: https://expressjs.com/ (접속: 2026-03-14)

[9] Node.js Foundation, "Node.js Documentation." [온라인] Available: https://nodejs.org/docs/latest-v22.x/api/ (접속: 2026-03-14)